/spec

Specification management: /spec create|add|delete|list|search|validate|status|task|govcreate|help

Overview

/spec manages the full lifecycle of specification documents stored under the project's specs/ directory. Each spec is a folder containing a SPEC.md (requirements and tasks), optional PLAN.md and TESTPLAN.md artifacts, and status metadata that /spec tracks through draft, review, approved, implemented, verified, and archived states. The command family covers creation, searching, validation, task tracking, JTBD analysis, and lifecycle transitions. Every form reports results in the message window as a table or status block.

Syntax

```
/spec                          # list active specs (default when no args)
/spec help                     # show the subcommand table
/spec create <name>            # scaffold a new spec folder
/spec govcreate <id> <ref> <dir>  # spec + scaffold from an architecture document
/spec add <name>               # add a spec discovered on disk into the registry
/spec delete <name>            # delete a spec
/spec validate <name>          # validate spec structure and requirements
/spec list [--status <s>] [--prefix <p>]
/spec search <query>           # keyword search across spec titles and content
/spec status <name> [<new-status>]
/spec task <name>              # task operations for a spec
/spec activate <name>          # mark a spec as the active spec
/spec deactivate               # clear the active spec
/spec coverage <name>          # requirement-to-task coverage report
/spec impl|implement <name> [--task <ID>] [--dry-run]
/spec jtbd <name> [--force] [--agent <name>]
/spec update <name>            # regenerate PLAN.md / TESTPLAN.md from SPEC.md
/spec specify <name>           # edit or regenerate the SPEC.md body
/spec plan <name>              # regenerate PLAN.md
/spec tasks <name>             # regenerate TESTPLAN.md
/spec feedback <name>          # manage review feedback (REVIEW.md / FEEDBACK.md)
```

Subcommands

Form	Description
/spec (bare)	Lists specs in the current status view.
/spec help	Prints the subcommand table.
/spec create <name>	Creates a new spec folder with a starter SPEC.md.
/spec govcreate <specid> <content-ref> <target-folder>	Authors a draft spec from architecture documentation (web URL or local file/folder) and scaffolds the project with the /new engine (see the govcreate section below).
/spec add <name>	Registers an existing spec folder found on disk.

Form	Description
`/spec delete <name>`	Removes the spec from the registry and disk.
`/spec validate <name>`	Checks structure, requirement IDs, and task links.
`/spec list [--status <s>] [--prefix <p>]`	Lists specs, filtered by status or name prefix.
`/spec search <query>`	Keyword search across titles and content.
`/spec status <name>`	Shows the spec's lifecycle status.
`/spec status <name> <s>`	Transitions the status (draft, in_review, approved, in_progress, implemented, verified, archived).
`/spec task <name>`	Task-tracking operations for the spec's plan.
`/spec activate <name>`	Sets the active spec used by /spec update, /spec tasks, and spec task tools.
`/spec deactivate`	Clears the active spec.
`/spec coverage <name>`	Renders the `[ok]`/`[wait]`/`[sync]`/`[stop]` coverage report linking requirements to completed tasks.
`/spec impl <name>`	Marks the spec as implementing; creates-or-updates session tracker tasks seeded from PLAN.md. `--task <ID>` targets one task, `--dry-run` previews without writing.
`/spec implement <name>`	Alias of `/spec impl`, same flags.
`/spec jtbd <name>`	Performs a Jobs-To-Be-Done analysis on the spec. `--force` overwrites prior analysis; `--agent <name>` runs the analysis through a specific agent.
`/spec update <name>`	Regenerates PLAN.md and TESTPLAN.md from an edited SPEC.md.
`/spec specify <name>`	Regenerates or edits the SPEC.md content itself.
`/spec plan <name>`	Regenerates PLAN.md only.
`/spec tasks <name>`	Regenerates TESTPLAN.md only.
`/spec feedback <name>`	Manages the spec's feedback loop (REVIEW.md / FEEDBACK.md).

`/spec govcreate <specid> <content-ref> <target-folder>`

Authors a new draft spec from architecture documentation and scaffolds the project in one step: the command acquires the referenced content, extracts the architecture structure, writes `SPEC.md` + `PLAN.md` + `TESTPLAN.md` into `<target-folder>/specs/<specid>/`, and scaffolds the project with the `/new` engine.

Arguments are positional and strictly ordered: all three positionals first, then the flags.

Argument	Meaning
`<specid>`	The new spec identifier. Alphanumeric with hyphens or underscores only, and must not start with `--`. The same validation rules as every other spec ID apply; a path-traversal name is rejected before any filesystem access.

Argument	Meaning
`<content-ref>`	Location of the documentation: a web URL to crawl, or a local file/folder path.
`<target-folder>`	The project folder to create or reuse. The spec is written to `<target-folder>/specs/<specid>/`.

Flags come after the positionals. The `/new` flags behave exactly as in `/new`; `--force` is govcreate-specific.

Flag	Default	Meaning
`--language <lang>`	`rust`	One of the languages accepted by `/new`.
`--type <type>`	`cmdline`	One of `library`, `cmdline`, `tui`, `gui`, `webapp`.
`--stack <name>`	none	A framework stack available for the language (e.g. `--stack axum`).
`--github`	off	Create the project on GitHub (mutually exclusive with `--gitlab`).
`--gitlab`	off	Create the project on GitLab (mutually exclusive with `--github`).
`--force`	off	Overwrite an existing `<target-folder>/specs/<specid>/`. Without `--force`, an existing spec directory is not overwritten.

What the command does:

1. **Classifies and acquires the content reference.** A URL runs a bounded, same-domain crawl that honours `robots.txt`, refuses private/loopback targets (SSRF protection) and non-HTTP(S) schemes, and never follows symlinks. A local file must be a supported document format (Markdown/text, PDF, DOCX, ODT, XLSX, ODS, PPTX, ODP, CSV, HTML, EPUB); a local folder is walked recursively for supported files. The crawl is bounded by page, depth, character, and deadline caps and reports the first cap reached (for example `pages`: 100 or `chars`) in the run notes.
2. **Extracts the architecture structure** — components, interfaces, data stores, external dependencies, and relationships — from the gathered text, falling back to a deterministic per-source structure when the model output is not usable.
3. **Authors the spec** with status `draft` frontmatter recording the source reference, `generated_by: govcreate`, and the scaffold options, then **scaffolds the project** via the `/new` engine in the target folder; `--github`/`--gitlab` create and push a remote repository.

Progress streams live in the message window and log panel while the run is in flight.

Failure reporting is explicit: an invalid spec ID or flag shows the usage block with the cause; a blocked or unreadable content reference explains why (SSRF, robots, unreadable/unsupported/escaping file); an

empty corpus or unusable model output stops before writing anything and names the cause; and an existing spec directory refuses unless `--force` was passed.

The scaffold step reuses the `/new` engine and its folder-layout rules unchanged; `/new` flags that are invalid for the chosen language are rejected with the same errors `/new` would produce.

Examples

`/spec create auth-refactor`

Creates the `specs/auth-refactor/` folder with a starter SPEC.md.

`/spec list --status in_review --prefix auth`

Lists in-review specs whose names start with `auth`.

`/spec coverage auth-refactor`

Shows which requirements are linked to completed tasks and which are pending.

`/spec implement auth-refactor --dry-run`

Previews the implementation task update without touching tracker state.

`/spec jtbd auth-refactor --agent general`

Runs a Jobs-To-Be-Done analysis on the spec through the general agent.

`/spec update auth-refactor`

Regenerates PLAN.md and TESTPLAN.md after SPEC.md edits. A spec with only a `SPEC.md` (no plan yet) is a valid target too: the prompt then omits the "existing plan" reference block, because there is nothing to preserve.

`/spec govcreate payments-arch https://docs.provider.gov/payment-service ./payments-svc`

Crawls the provider's architecture pages and creates the `payments-arch` draft spec plus a Rust cmdline scaffold in `./payments-svc`.

`/spec govcreate records-arch ./arch-records/records-hld.pdf ./records --language python --type li`

Reads the local PDF and creates the `records-arch` draft spec plus a Python library scaffold in `./records`.

`/spec govcreate billing-arch ./doc/sad.md ./billing-svc --language rust --type cmdline --stack ax`

Reads the local Markdown design document and creates the draft spec plus an axum-based Rust scaffold; fails if `billing-svc/specs/billing-arch/` already exists unless `--force` is added.

Output

- Lists render as spec tables with name, status, and metadata columns.
- Coverage reports use `[ok]` (requirement covered by completed tasks), `[wait]` (tasks pending), `[sync]` (drift between spec and tracker), and `[stop]` (blocked) task-status symbols.
- Lifecycle transitions print a confirmation with the old and new status.
- Validation failures list each problem with a pointer to the offending requirement or task ID.
- Spec file writes are atomic (temp file plus rename). Writing an empty REVIEW.md or FEEDBACK.md clears it. Task-completion heuristics only fire for writes inside the active spec directory.

Related

- `/spec coverage` semantics are shared with the spec_task_update tool.
- `ragent-specs` crate implements the lifecycle engine behind every form.
- specs//SPEC.md is the source of truth; PLAN.md and TESTPLAN.md are derived artifacts.
- See docs/howtos/ for the spec system manual and JTBD analysis guide.